

بسم الله الرحمن الرحيم

گزارش کار پروژه درس ساختار و زبان کامپیوتر

مبین روان نخبجوانی

۴۰۲۱۰۶۰۶۱

پاییز ۱۴۰۳

در این پروژه به پیاده سازی بازی پونگ (**Pong**) پرداختیم و هدف اصلی پروژه این است که نشان دهیم در صورت انجام محاسبات با اسمبلی میتوانیم به سود سرعت یک معامله خوب داشته باشیم.

بازی به این صورت است که توپی در صفحه در حال حرکت هست و شما در مقابل کامپیوتر باید بتوانید با استفاده از صفحه های کوچکی که در اختیار دارید (**Paddle**) از خارج شدن توپ از صفحه جلوگیری کنید.

توضیحات نحوه اجرای بازی:

- در گام اول کتابخانه های مربوطه را بارگذاری میکنیم

```
#include <SFML/Graphics.hpp> // Include the SFML graphics library for rendering
#include <SFML/Audio.hpp>    // Include the SFML audio library for sound
#include <cmath>              // Include the math library for mathematical functions
#include <iostream>           // Include the input/output stream library
#include <bits/stdc++.h>      // Include the standard C++ library (all headers)
#include <chrono>             // Include the chrono library for time measurement
using namespace std;        // Use the standard namespace
```

در کامنت های روبه روی کد ها میتوانید علت استفاده از هر کتابخانه را مشاهده کنید

- تعریف کردن ثابت (Constant) های مورد نیاز:

```
// Define constants for the game
const int WINDOW_WIDTH = 800;    // Width of the game window
const int WINDOW_HEIGHT = 600;   // Height of the game window
const float PADDLE_WIDTH = 10.f; // Width of the paddle
const float PADDLE_HEIGHT = 100.f; // Height of the paddle
const float BALL_RADIUS = 10.f;  // Radius of the ball
const float PADDLE_SPEED = 5.0f; // Speed of the player's paddle
const float BALL_SPEED_BASE = 5.0f; // Base speed of the ball
const float AI_PADDLE_SPEED_BASE = 0.6f; // Base speed of the AI's paddle
```

- ساختن کلاس (Class) های مورد نیاز

○ یک Enum برای کنترل وضعیت حرکت توپ که با کلید های ۱ یا ۲ یا

۳ در حین بازی قابل تغییر است:

```
// Enum to define different ball movement modes
enum BallMode {
    Straight, // Ball moves in a straight line
    Sinusoidal, // Ball moves in a sinusoidal pattern
    Parabolic // Ball moves in a parabolic pattern
};
```

○ یک کلاس برای کنترل توپ:

```
51 class Ball {
52 public:
53     sf::CircleShape shape; // SFML circle shape for the ball
54     sf::Vector2f velocity; // Velocity of the ball
55     sf::RectangleShape crossHorizontal; // Horizontal cross on the ball
56     sf::RectangleShape crossVertical; // Vertical cross on the ball
57     float rotationAngle; // Rotation angle of the cross
58
59     // Constructor to initialize the ball
60     Ball(float x, float y, float speed) {
61         shape.setRadius(BALL_RADIUS); // Set ball radius
62         shape.setPosition(x, y); // Set ball position
63         shape.setFillColor(sf::Color::Yellow); // Set ball color
64         velocity = sf::Vector2f(speed, 0); // Set initial velocity
65
66         // Initialize the horizontal cross
67         crossHorizontal.setSize(sf::Vector2f(BALL_RADIUS * 1.5f, 2.f));
68         crossHorizontal.setFillColor(sf::Color::Black);
69         crossHorizontal.setOrigin(crossHorizontal.getSize().x / 2, crossHorizontal.getSize().y / 2);
70
71         // Initialize the vertical cross
72         crossVertical.setSize(sf::Vector2f(2.f, BALL_RADIUS * 1.5f));
73         crossVertical.setFillColor(sf::Color::Black);
74         crossVertical.setOrigin(crossVertical.getSize().x / 2, crossVertical.getSize().y / 2);
75
76         rotationAngle = 0.f; // Initialize rotation angle
77     }
78
79     // Function to move the ball
80     void move() {}
81     shape.move(velocity); // Move the ball
82
83     // Bounce the ball off the top and bottom walls
84     if (shape.getPosition().y < 0 || shape.getPosition().y + 2 * BALL_RADIUS > WINDOW_HEIGHT) {
85         velocity.y = -velocity.y;
86     }
87
88
89     // Function to reset the ball position and velocity
90     void reset(float speed) {
91         shape.setPosition(WINDOW_WIDTH / 2, WINDOW_HEIGHT / 2); // Reset position to center
92         velocity = sf::Vector2f(speed, 0); // Reset velocity
93     }
94
95     // Function to update the position of the cross on the ball
96     void updateCrossPosition() {
97         sf::Vector2f ballCenter = shape.getPosition() + sf::Vector2f(BALL_RADIUS, BALL_RADIUS); // Calculate ball center
98         crossHorizontal.setPosition(ballCenter); // Set horizontal cross position
99         crossVertical.setPosition(ballCenter); // Set vertical cross position
100     }
101
102     // Function to rotate the cross on the ball
103     void rotateCross(float angle) {
104         crossHorizontal.rotate(angle); // Rotate horizontal cross
105         crossVertical.rotate(angle); // Rotate vertical cross
106     }
107 };
108
```

در کانستراکتور آن یک توپ با شعاع در لوکیشن معلوم با سرعت افقی معلوم ساخته میشود همچنین برای نشان دادن چرخش توپ دو مستطیل که به صورت یک بعلاوه روی مرکز توپ قرار میگیرند ساخته میشوند

در تابع **move** توپ بر اساس سرعتی که دارد حرکت میکند و از توابع

خود **cpp** استفاده شده

تابع ریست برای این است که توپ دوباره با همان سرعت پایه ای به مرکز صفحه برگردد

تابع **updateCrossPosition** تضمین میکند که بعلاوه ای که

روی توپ قرار داریم همواره با توپ حرکت کند

تابع **rotateCross** بعلاوه ای که روی توپ قرار دادیم را در صورت

حرکت سینوسی یا پرتابی توپ میچرخاند

○ کلاس برای کنترل صفحه های حرکتی (Paddle):

```
// Class representing the paddle
class Paddle {
public:
    sf::RectangleShape shape; // SFML rectangle shape for the paddle
    float speed;              // Speed of the paddle

    // Constructor to initialize the paddle
    Paddle(float x, float y, float mySpeed) {
        shape.setSize(sf::Vector2f(PADDLE_WIDTH, PADDLE_HEIGHT)); // Set paddle size
        shape.setPosition(x, y); // Set paddle position
        speed = mySpeed;        // Set paddle speed
    }

    // Function to move the paddle
    void move(float offset) {
        shape.move(0, offset * speed); // Move the paddle
        // Prevent the paddle from moving out of the window
        if (shape.getPosition().y < 0)
            shape.setPosition(shape.getPosition().x, 0);
        if (shape.getPosition().y + PADDLE_HEIGHT > WINDOW_HEIGHT)
            shape.setPosition(shape.getPosition().x, WINDOW_HEIGHT - PADDLE_HEIGHT);
    }
};
```

در کانستراکتور ابتدا اندازه و مکان پدل ها تنظیم میشوند و سرعت حرکت آن مشخص میشود

و در تابع **move** با توجه به مقدار داده شده پدل جا به جا میشود و کنترل میشود که پدل از بالا یا پایین صفحه خارج نشود

• **Initialize** کردن لوازم مورد نیاز برای شروع بازی:

```

int main() {
    // Prompt user to select a game level
    int level = 1;
    std::cout << "Select a level (1 to 10): ";
    std::cin >> level;

    // Clamp the level between 1 and 10
    if (level < 1) level = 1;
    if (level > 10) level = 10;

    // Calculate AI paddle speed based on level
    const float AI_PADDLE_SPEED = AI_PADDLE_SPEED_BASE * level + 5;
    // Set ball speed based on level
    float temp_speed = (level <= 5) ? 10 : 13;
    const float BALL_SPEED = temp_speed;

    // Create the game window
    sf::RenderWindow window(sf::VideoMode(WINDOW_WIDTH, WINDOW_HEIGHT), "Pong Game");
    window.setFramerateLimit(60); // Set frame rate limit to 60 FPS

    // Load background texture
    sf::Texture backgroundTexture;
    if (!backgroundTexture.loadFromFile("/home/mobin/Pictures/pong.png")) {
        std::cerr << "Failed to load background image!" << std::endl;
        return -1;
    }
    sf::Sprite backgroundSprite(backgroundTexture); // Create background sprite

    // Initialize paddles and ball
    Paddle playerPaddle(10, WINDOW_HEIGHT / 2 - PADDLE_HEIGHT / 2, PADDLE_SPEED);
    Paddle aiPaddle(WINDOW_WIDTH - 20, WINDOW_HEIGHT / 2 - PADDLE_HEIGHT / 2, AI_PADDLE_SPEED);
    Ball ball(WINDOW_WIDTH / 2, WINDOW_HEIGHT / 2, BALL_SPEED);

    // Initialize scores
    int playerScore = 0;
    int aiScore = 0;

    // Load font for score display
    sf::Font font;
    if (!font.loadFromFile("/home/mobin/Documents/Fonts/ARIAL.TTF")) {
        std::cerr << "Failed to load font!" << std::endl;
        return -1;
    }
}

```

در ابتدا سطح بازی توسط کاربر تعیین میشود و سرعت حرکت توپ و پدل با توجه به سطح تعیین میشود و یک پنجره ساخته میشود و پس زمینه آن بالا می آید و فونت و موسیقی پس زمینه بالا می آیند و امتیاز ها ساخته میشوند... در گام بعدی در یک حلقه تک تک اتفاقاتی که در طول بازی رخ میدهد را بررسی میکنیم اما با توجه به اینکه هدف مقایسه زمانی اسمبلی و بدون اسمبلی است ما

قبل از شروع حلقه یک زمان گیر را شروع میکنیم و بعد از حلقه آن را تمام میکنیم و با یک شمارنده تعداد دفعاتی که در حلقه گذر کردیم را میشماریم و در نهایت میانگین زمان یک حلقه را به عنوان زمان گزارش میکنیم:

```
// Start time measurement
auto start = std::chrono::high_resolution_clock::now();
float counter = 0; // Counter for time measurement

// Main game loop
// we count count of times we went through this loop with counter
while (window.isOpen()) { ...
}

// Calculate and print the average time taken for ball movement and collision
auto end = std::chrono::high_resolution_clock::now();
auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end - start).count() / counter;
std::cout << "Time taken for ball movement and collision: " << duration << " microseconds" << std::endl;
```

• حلقه اصلی بازی:

○ حرکت پدل ها و چک کردن وضعیت امتیاز ها

```
// Main game loop
// we count count of times we went through this loop with counter
while (window.isOpen()) {
    sf::Event event;
    while (window.pollEvent(event)) {
        if (event.type == sf::Event::Closed)
            window.close(); // Close the window if the close event is triggered
    }

    // Check if the game is over (either player or AI reaches 5 points)
    if (playerScore >= 5 || aiScore >= 5) {
        std::string winner = (playerScore >= 5) ? "Mobin" : "AI";
        std::cout << "Game Over! " << winner << " wins!" << std::endl;
        break; // End the game loop
    }

    // Player movement
    if (sf::Keyboard::isKeyPressed(sf::Keyboard::Up))
        playerPaddle.move(-1); // Move player paddle up
    if (sf::Keyboard::isKeyPressed(sf::Keyboard::Down))
        playerPaddle.move(1); // Move player paddle down

    // AI movement
    float aiPaddleCenter = aiPaddle.shape.getPosition().y + PADDLE_HEIGHT / 2;
    float ballFutureY = ball.shape.getPosition().y + ball.velocity.y * (aiPaddle.shape.getPosition().x - ball.shape.getPosition().x) / ball.velocity.x;

    if (aiPaddleCenter < ballFutureY) {
        aiPaddle.move(1); // Move AI paddle down
    } else {
        aiPaddle.move(-1); // Move AI paddle up
    }
}
```

در ابتدای وایل چک میکنیم که اگر بازی بسته شده پنجره را ببندد سپس چک میکنیم که اگر امتیاز کسی به ۵ رسیده بازی را تمام کند و در نهایت

حرکت پدل ها را کنترل میکنیم که شایان ذکر است که این حرکت در نسخه سریعتر بازی به صورت موازی نیز پیاده سازی شده:

```
// Parallelize paddle movement using OpenMP
#pragma omp parallel sections
{
    #pragma omp section
    {
        // Move player paddle up or down based on keyboard input
        if (sf::Keyboard::isKeyPressed(sf::Keyboard::Up))
            playerPaddle.move(-1);
        if (sf::Keyboard::isKeyPressed(sf::Keyboard::Down))
            playerPaddle.move(1);
    }

    #pragma omp section
    {
        // Move AI paddle based on ball position
        float aiPaddleCenter = aiPaddle.shape.getPosition().y + PADDLE_HEIGHT / 2;
        float ballFutureY = ball.shape.getPosition().y + ball.velocity.y * (aiPaddle.shape.getPosition().x - ball.shape.getPosition().x) / ball.velocity.x;

        if (aiPaddleCenter < ballFutureY) {
            aiPaddle.move(1);
        } else {
            aiPaddle.move(-1);
        }
    }
}
```

○ رسیدگی کردن به حالت های مختلف حرکت توپ:

```
170 int main() {
171     while (window.isOpen()) {
207         // Ball mode control
208         if (sf::Keyboard::isKeyPressed(sf::Keyboard::Num1)) {
209             ballMode = Straight; // Set ball mode to straight
210         }
211         if (sf::Keyboard::isKeyPressed(sf::Keyboard::Num2)) {
212             ballMode = Sinusoidal; // Set ball mode to sinusoidal
213         }
214         if (sf::Keyboard::isKeyPressed(sf::Keyboard::Num3)) {
215             ballMode = Parabolic; // Set ball mode to parabolic
216         }
217
218         // Apply ball movement based on the selected mode
219         switch (ballMode) {
220             case Straight:
221                 break;
222             case Sinusoidal:
223                 ball.velocity.y = std::sin(ball.shape.getPosition().x * 0.01f) * BALL_SPEED; // Set vertical velocity using sine function
224                 // Handle ball collision with top and bottom walls
225                 if (ball.shape.getPosition().y < 0) {
226                     ball.shape.setPosition(ball.shape.getPosition().x, 0);
227                     ball.velocity.y = std::abs(ball.velocity.y); // Reverse vertical velocity
228                 }
229                 if (ball.shape.getPosition().y + 2 * BALL_RADIUS > WINDOW_HEIGHT) {
230                     ball.shape.setPosition(ball.shape.getPosition().x, WINDOW_HEIGHT - 2 * BALL_RADIUS);
231                     ball.velocity.y = -std::abs(ball.velocity.y); // Reverse vertical velocity
232                 }
233                 ball.rotateCross(2.f); // Rotate the cross on the ball
234                 break;
235             case Parabolic:
236                 ball.velocity.y += 8.1f; // Simulate gravity effect
237                 // Handle ball collision with top and bottom walls
238                 if (ball.shape.getPosition().y < 0) {
239                     ball.shape.setPosition(ball.shape.getPosition().x, 0);
240                     ball.velocity.y = std::abs(ball.velocity.y) + 1; // Reverse vertical velocity
241                 }
242                 if (ball.shape.getPosition().y + 2 * BALL_RADIUS > WINDOW_HEIGHT) {
243                     ball.shape.setPosition(ball.shape.getPosition().x, WINDOW_HEIGHT - 2 * BALL_RADIUS);
244                     ball.velocity.y = -std::abs(ball.velocity.y) - 1; // Reverse vertical velocity
245                 }
246                 ball.rotateCross(2.f); // Rotate the cross on the ball
247                 break;
248         }
249
250         // Move the ball
251         ball.move();
    }
```

در رابطه با رسیدگی به حالات مختلف حرکتی توپ چهار نکته مورد توجه است:

1. محاسبه خود سینوس مکان افقی توپ برای حرکت عمودی توپ که در نسخه سریعتر با اسمبلی به دو صورت میتواند پیاده سازی شود:

```
// Function to calculate sine using assembly (inline assembly)

float sin_asm(float x) {
    float result;
    asm __volatile__ (
        "fsin" // Use the x86 fsin instruction to calculate sine
        : "=t" (result) // Output result
        : "0" (x) // Input x
    );
    return result;
}
```

```
// Alternative function for sin using SIMD (commented out)

m128 sin_simd( __m128 x) {
    __m128 x3 = _mm_mul_ps(x, _mm_mul_ps(x, x)); // x^3
    __m128 x5 = _mm_mul_ps(x3, _mm_mul_ps(x, x)); // x^5
    __m128 x7 = _mm_mul_ps(x5, _mm_mul_ps(x, x)); // x^7
    __m128 x9 = _mm_mul_ps(x7, _mm_mul_ps(x, x)); // x^9

    __m128 sin_result = _mm_add_ps(
        _mm_sub_ps(
            _mm_add_ps(
                _mm_sub_ps(x, _mm_div_ps(x3, _mm_set1_ps(6.0f))),
                _mm_div_ps(x5, _mm_set1_ps(120.0f))
            ),
            _mm_div_ps(x7, _mm_set1_ps(5040.0f))
        ),
        _mm_div_ps(x9, _mm_set1_ps(362880.0f))
    );
    return sin_result;
}
```

دومین روش پیاده سازی سینوس از سری تیلور و رجیسترهای برداری

استفاده کرده که جملات به صورت موازی محاسبه و با یکدیگر جمع یا از هم کم میشوند در روش اول از اسمبلی اینلاین استفاده شده که یک فلو ت به

عنوان تایپ خروجی تعریف میشود *volatile* به دو منظور استفاده

می شود:

اثرات جانبی: در برخی موارد، دستورات اسمبلی ممکن است به طور مستقیم با سخت افزار یا حافظه خاصی کار کنند و تغییراتی ایجاد کنند که برای

کامپایلر قابل تشخیص نیست. استفاده از *volatile* تضمین می کند که این دستورات حذف نشوند یا ترتیب آنها تغییر نکند.

بهینه سازی کامپایلر: کامپایلرها معمولاً کد را بهینه سازی می کنند و ممکن است دستوراتی که به نظر آنها بی اثر است را حذف کنند. اما در مورد دستورات اسمبلی، این بهینه سازی ممکن است باعث ایجاد خطا شود.

volatile به کامپایلر می گوید که این دستورات را بهینه سازی نکند.

در خط اول تابعی که میخواهد صدا شود را مینویسیم و با تگ های "O" یا "I" یا "2" یا ... ورودی ها را میدهیم و با تگ هایی مانند "t=" یا "u=" یا ... خروجی ها را میگیریم.

• نحوه صدا کردن توابع:

○ تابع $\sin$:

```
float sin_value = sin_asm(ball.shape.getPosition().x * 0.01);  
ball.velocity.y = sin_value * BALL_SPEED;
```

○ تابع با بسط تیلور:

```
float input_value = ball.shape.getPosition().x * 0.006f;  
__m128 input_sind = __mm_set1_ps(input_value);  
__m128 result_sind = sin_sind(input_sind);  
  
float sin_value;  
__mm_store_ss(&sin_value, result_sind);  
  
ball.velocity.y = sin_value * BALL_SPEED;
```

ابتدا با تابع `__mm_set1_ps` مقادیر فلوت را به صورت برداری ذخیره میکنیم و با `__mm_store_ss` مقدار دریافتی برداری را به صورت فلوت ذخیره میکنیم (توجه کنید که به دلیل شکل تابع مقدار کوچکتري از مقداری که به سینوس عادی میدهیم را باید به آن پاس بدهیم)

2. برخورد با سقف و زمین:

```
if (ball.shape.getPosition().y < 0) {  
    if(level >= 7) ball.shape.setPosition(ball.shape.getPosition().x, WINDOW_HEIGHT - 2 * BALL_RADIUS);  
    else {  
        ball.velocity.y = (ball.velocity.y>0) ? ball.velocity.y : -ball.velocity.y;  
        ball.shape.setPosition(ball.shape.getPosition().x, 0);  
    }  
}  
  
if (ball.shape.getPosition().y + 2 * BALL_RADIUS > WINDOW_HEIGHT) {  
    if(level >= 7) ball.shape.setPosition(ball.shape.getPosition().x, 0);  
    else {  
        ball.shape.setPosition(ball.shape.getPosition().x, WINDOW_HEIGHT - 2 * BALL_RADIUS);  
        ball.velocity.y = (ball.velocity.y<0) ? ball.velocity.y : -ball.velocity.y;  
    }  
}  
ball.rotateCross(2.f); // Rotate the cross on the ball  
break;
```

در سطوح بالای ۷ در صورت برخورد با سقف یا زمین توپ از آن طرف باز میگردد و در لول های پایین تر از ۷ توپ بعد از برخورد از همانجا می آید

3. چرخش توپ در حالات سینوسی و پرتابی که با تابع `rotateCross` همانطور

که در بخش معرفی کلاس ها توضیح داده شده بود با سرعت زاویه ای معلوم

پیاده سازی شده

• برخورد با پدل ها:

```
// Handle ball collision with paddles
if (ball.shape.getGlobalBounds().intersects(playerPaddle.shape.getGlobalBounds())) {
    ball.velocity.x = -ball.velocity.x; // Reverse ball direction
    float hitPoint = (ball.shape.getPosition().y + BALL_RADIUS) - (playerPaddle.shape.getPosition().y + PADDLE_HEIGHT / 2);
    ball.velocity.y = hitPoint * 0.1f; // Adjust ball velocity based on where it hits the paddle
}
if (ball.shape.getGlobalBounds().intersects(aiPaddle.shape.getGlobalBounds())) {
    ball.velocity.x = -ball.velocity.x; // Reverse ball direction
    float hitPoint = (ball.shape.getPosition().y + BALL_RADIUS) - (aiPaddle.shape.getPosition().y + PADDLE_HEIGHT / 2);
    ball.velocity.y = hitPoint * 0.1f; // Adjust ball velocity based on where it hits the paddle
}
```

دو نکته دارد که یکی سرعت افقی توپ را قرینه میکند و دیگری بر اساس نقطه

برخورد یک سرعت عمودی به توپ میدهد و دوباره سرعت توپ بر اساس نوع

حرکت محاسبه میشود

• برخورد نکردن با پدل ها:

```
// Handle ball going out of bounds (left or right)
if (ball.shape.getPosition().x < 0) {
    aiScore++; // Increment AI score
    ball.reset(BALL_SPEED); // Reset ball position and velocity
}
if (ball.shape.getPosition().x + 2 * BALL_RADIUS > WINDOW_WIDTH) {
    playerScore++; // Increment player score
    ball.reset(BALL_SPEED); // Reset ball position and velocity
}
```

اگر پدلی نتواند توپ را بگیرد امتیازی حریف یکی بیشتر میشود و توپ دوباره به

مرکز زمین باز میگردد

- بارگذاری بازی:

```
// Update score text
scoreText.setString(std::to_string(playerScore) + " - " + std::to_string(aiScore));

// Update cross position on the ball
ball.updateCrossPosition();

// Clear the window and draw all game objects
window.clear();
window.draw(backgroundSprite); // Draw background
window.draw(playerPaddle.shape);
window.draw(aiPaddle.shape);
window.draw(ball.shape);
window.draw(ball.crossHorizontal); // Draw horizontal cross
window.draw(ball.crossVertical); // Draw vertical cross
window.draw(scoreText);
window.display();

counter++; // Increment counter for time measurement
}
```

بعد از برخورد امتیازها به روز رسانی میشوند و مرکز بعلاوه روی توپ در مرکز توپ قرار میگیرد و سپس مواد مورد نیاز پیاده سازی بازی پیاده سازی میشوند و در نهایت شمارنده برای شمردن تعداد حرکت کردن ها در وایل مقایسه سرعت:

-

ابتدا در فایل زیر حالت سینوسی با اسمبلی ($\sin$) و بدون اسمبلی مقایسه شده که حالت اسمبلی حدود ۱۰۰ میکروثانیه سریعتر است:

```
~/Documents/Sakhtar-project/assembly > g++ -o pong project.cpp -lsfml-audio -lsfml-graphics -lsfml-window -lsfml-system
~/Documents/Sakhtar-project/assembly > ./pong
Select a level (1 to 10): 5
Game Over! AI wins!
Time taken for ball movement and collision: 16754.3 microseconds
~/Documents/Sakhtar-project/assembly > cd ../no\_assembly
~/Documents/Sakhtar-project/no assembly > g++ -o pong project.cpp -lsfml-audio -lsfml-graphics -lsfml-window -lsfml-system
~/Documents/Sakhtar-project/no assembly > ./pong
Select a level (1 to 10): 5
Game Over! AI wins!
Time taken for ball movement and collision: 16859.4 microseconds
```

و در فایل زیر اسمبلی سینوس با تیلور هم مقایسه شده (سومین زمان گزارش شده)


```
~/Documents/Sakhtar-project/no assembly > g++ -o pong project.cpp -lsfml-audio -lsfml-graphics -lsfml-window -lsfml-system
~/Documents/Sakhtar-project/no assembly > ./pong
Select a level (1 to 10): 5
Game Over! Mobin wins!
Time taken for ball movement and collision: 16864.8 microseconds
~/Documents/Sakhtar-project/no assembly > cd ../assembly
~/Documents/Sakhtar-project/assembly > g++ -o pong project.cpp -lsfml-audio -lsfml-graphics -lsfml-window -lsfml-system
~/Documents/Sakhtar-project/assembly > ./pong
Select a level (1 to 10): 5
Game Over! AI wins!
Time taken for ball movement and collision: 16711.2 microseconds
~/Documents/Sakhtar-project/assembly > g++ -o pong project.cpp -lsfml-audio -lsfml-graphics -lsfml-window -lsfml-system
~/Documents/Sakhtar-project/assembly > ./pong
Select a level (1 to 10): 5
Game Over! AI wins!
Time taken for ball movement and collision: 16779.4 microseconds
```

در نهایت با توجه به اینکه فقط محاسبات سینوس دارد به چند روش انجام میشود
افزایش سرعت خیلی نیست اما قطعا استفاده از اسمبلی در پروژه های بزرگتر
قابلیتی است که به عنوان آورده این پروژه در ذهن خواهد ماند.

و منسره الله توفیق
میسره روان نخوانسره